

Raport laboratoryjny

Planowanie z wykorzystaniem języka PDDL

Przedmiot: Sztuczna Inteligencja i Inżynieria Wiedzy – Lista nr 3

Zakres: Modelowanie problemów planowania w PDDL, uruchomienie planerów, analiza wygenerowanych planów (długość, koszt, czas trwania, topologia).

Autor: Dawid Błaszczuk

Spis treści

1. [Wstęp teoretyczny](#)
 - [1.1. Czym jest PDDL?](#)
 - [1.2. Model STRIPS](#)
2. [Zadanie 1 – Transport intermodalny paczek \(25 pkt\)](#)
 - [3.1. Model logiczny – domena](#)
 - [3.2. Opis problemu](#)
 - [3.3. Topologia sieci transportowej](#)
 - [3.4. Wynik planera](#)
 - [3.5. Analiza planu](#)
3. [Zadanie 2 – Robot odkurzający \(15 pkt\)](#)
 - [4.1. Opis problemu](#)
 - [4.2. Plik domain.pddl](#)
 - [4.3. Plik problem.pddl](#)
 - [4.4. Wygenerowany plan](#)
 - [4.5. Analiza](#)
4. [Zadanie 3 – Robot z dwoma ramionami \(10 pkt\)](#)
 - [5.1. Opis problemu](#)
 - [5.2. Domena i problem \(zgodne z listą\)](#)
 - [5.3. Wygenerowany plan](#)
 - [5.4. Analiza planu](#)

- 5. [Porównanie eksperymentów](#)
 - 6. [Wnioski końcowe](#)
-

1. Wstęp teoretyczny

1.1. Czym jest PDDL?

PDDL (*Planning Domain Definition Language*) to standardowy język opisu problemów planowania klasycznego. Oddziela on:

- **domenę** (`domain.pddl`) – typy obiektów, predykaty (fakty), schematy akcji (operatorów);
- **problem** (`problem.pddl`) – konkretne obiekty, stan początkowy (`:init`) i cel (`:goal`).

Planer szuka sekwencji **zinstancjonowanych** akcji prowadzącej ze stanu początkowego do stanu spełniającego cel.

1.2. Model STRIPS

W podstawowym wariantcie (**strips**, wymaganie `:strips`) każda akcja ma:

- **warunki wstępne** (*preconditions*) – fakty, które muszą być prawdziwe przed wykonaniem;
- **efekty** – listę faktów do dodania i usunięcia (`not`).

Stan planowania to zbiór faktów (założenie zamkniętego świata). Operator jest **dopuszczalny**, gdy wszystkie jego preconditions są w bieżącym stanie; po zastosowaniu stan jest aktualizowany zgodnie z efektami.

Rozszerzenia używane w tej liście:

Rozszerzenie	Zastosowanie w projekcie
<code>:typing</code>	Typy <code>robot</code> , <code>room</code> , <code>package</code> , <code>truck</code> , ...
<code>:negative-preconditions</code>	(<code>not (at ...)</code>) , (<code>not (dirty ...)</code>) w efektach i warunkach
<code>:durative-actions</code>	Akcje czasowe w Zadaniu 1 (ładowanie, jazda, lot; makespan)
<code>:strips</code>	Podstawa wszystkich trzech zadań

3. Zadanie 1 – Transport intermodalny paczek (25 pkt)

3.1. Model logiczny – domena

Wymagania domeny: `:strips` `:typing` `:durative-actions`.

Typy – hierarchia lokalizacji i pojazdów:

```
(:types
  location package vehicle - object
  warehouse airport port station - location
  truck plane ship train - vehicle
)
```

Predykaty kluczowe:

Predykat	Znaczenie
<code>(package-at ?p ?l)</code>	Paczka w lokacji
<code>(vehicle-at ?v ?l)</code>	Pojazd w lokacji
<code>(loaded ?p ?v)</code>	Paczka na pokładzie
<code>(vehicle-empty ?v)</code>	Pojazd bez ładunku
<code>(compatible ?p ?v)</code>	Dopuszczalne połączenie paczka-pojazd
<code>(road-connection ...)</code>	Droga
<code>(rail-connection ...)</code>	Kolej
<code>(sea-connection ...)</code>	Morze
<code>(flight-connection ...)</code>	Lot

Przykład akcji czasowej – załadunek (5 jednostek czasu):

```
(:durative-action load
  :parameters (?p - package ?v - vehicle ?l - location)
  :duration (= ?duration 5)
  :condition (and
    (at start (package-at ?p ?l))
    (at start (vehicle-at ?v ?l))
    (at start (compatible ?p ?v))
    (at start (vehicle-empty ?v))
    (over all (vehicle-at ?v ?l))
  )
  :effect (and
```

```

(at start (not (package-at ?p ?l)))
(at start (not (vehicle-empty ?v)))
(at end (loaded ?p ?v))
)
)

```

Przemieszczanie pojazdów – dla każdego typu pojazdu w domenie jest osobna akcja czasowa o **tym samym schemacie**: na początku pojazd stoi w `?from`, przez cały czas trwania musi obowiązywać właściwe połączenie w sieci (`over all`), na końcu pojazd jest w `?to`. Różnią się jedynie typem pojazdu, predykatem topologii i wartością `:duration`.

Przykład – jazda ciężarówki po sieci drogowej (`drive-truck`, 15 jednostek czasu):

```

(:durative-action drive-truck
 :parameters (?t - truck ?from - location ?to - location)
 :duration (= ?duration 15)
 :condition (and
  (at start (vehicle-at ?t ?from))
  (over all (road-connection ?from ?to))
 )
 :effect (and
  (at start (not (vehicle-at ?t ?from)))
  (at end (vehicle-at ?t ?to))
 )
)

```

Analogiczne akcje dla pozostałych środków transportu:

Akcja	Pojazd	Połączenie	Czas
<code>fly-plane</code>	samolot(<code>plane</code>)	<code>flight-connection</code> (lotnisko → lotnisko)	8
<code>sail-ship</code>	statek(<code>ship</code>)	<code>sea-connection</code> (port → port)	30
<code>move-train</code>	pociąg(<code>train</code>)	<code>rail-connection</code> (stacja → stacja)	20

Czasy trwania akcji – planer minimalizuje **makespan** (czas zakończenia ostatniej akcji):

Akcja	Czas (<code>:duration</code>)	Uwaga
<code>load / unload</code>	5	Operacje załadunku
<code>drive-truck</code>	15	Droga
<code>fly-plane</code>	8	Najszybszy odcinek lotniczy

Akcja	Czas (:duration)	Uwaga
sail-ship	30	Najwolniejszy, ale sensowny dla dużego ładunku
move-train	20	Kolej

3.2. Opis problemu

Cel: Zaplanować dostawę czterech paczek do wskazanych lokalizacji, korzystając z sieci transportu **drogowego, kolejowego, morskiego i lotniczego**, z uwzględnieniem **czasu trwania** operacji.

Obiekty (skrót):

- Magazyny: `poznan_magazyn`, `rzyszow_magazyn`
- Porty: `gdansk_port`, `szczecin_port`
- Lotniska: `warszawa_lotnisko`, `katowice_lotnisko`
- Stacje: `wroclaw_stacja`, `krakow_stacja`, `szczecin_stacja`
- Pojazdy: ciężarówki, pociąg, statek, samolot
- Paczki: `paczka_medyczna`, `elektronika`, `czesci_samochodowe`, `turbina_wiatrowa`

Cel planowania:

```
(:goal (and
  (package-at paczka_medyczna poznan_magazyn)
  (package-at elektronika rzyszow_magazyn)
  (package-at czesci_samochodowe rzyszow_magazyn)
  (package-at turbina_wiatrowa krakow_stacja)
))
```

Ograniczenie zgodności (`compatible`): np. `turbina_wiatrowa` nie może być ładowana do samolotu – wymusza trasę morską/kolejową/drogową.

3.3. Topologia sieci transportowej

Sieć jest **skierowana** (połączenia dwukierunkowe zdefiniowane osobno). Kluczowe ścieżki w instancji:

- Droga:** sieć ma charakter **gwiazdy** – magazyny i lotniska łączą się drogą z `warszawa_lotnisko` (główny węzeł przeładunkowy); dodatkowo istnieje bezpośredni odcinek `gdansk_port ↔ wroclaw_stacja`.
- Kolej:** stacje `wroclaw_stacja`, `krakow_stacja` i `szczecin_stacja` tworzą **trójkąt** – każda para jest połączona w obie strony.

- **Morze:** `gdansk_port ↔ szczecin_port`.
- **Lot:** `katowice_lotnisko ↔ warszawa_lotnisko` (samolot startuje w Katowicach).

Topologia wymusza **zmianę trybu transportu** w węzłach przeładunkowych (np. port → ciężarówka → stacja → pociąg).

3.4. Wynik planera

Parametr	Wartość
Pliki	<code>domain.pddl</code> , <code>problem.pddl</code>
Planer	OPTIC (lub planer temporalny; log w <code>Zadanie-1/plan.md</code>)
Metryka	makespan (czas zakończenia planu)
Makespan	95,002
Liczba akcji czasowych	22 (w tym równoległe od $t = 0$)
Ocenione stany	93
Czas planowania	~0,15 s

Fragment planu:

```
0.000: (move-train pociag_towarowy wroclaw_stacja szczecin_stacja)
[20.000]
0.000: (load turbina_wiatrowa kontenerowiec_baltyk gdansk_port)
[5.000]
0.000: (drive-truck tir_warszawa warszawa_lotnisko wroclaw_stacja)
[15.000]
0.000: (load elektronika tir_poznan poznan_magazyn)
[5.000]
5.000: (sail-ship kontenerowiec_baltyk gdansk_port szczecin_port)
[30.000]
...
90.002: (unload turbina_wiatrowa pociag_towarowy krakow_stacja)
[5.000]
```

Pełny log: `Zadanie-1/plan.md`.

3.5. Analiza planu

Poprawność

- Każda operacja `load / unload` wymaga obecności pojazdu i paczki w tej samej lokacji przez cały czas trwania(`over all`).
- `turbina_wiatrowa` : statek Gdańsk → Szczecin → ciężarówka → stacja → pociąg do Krakowa – zgodne z (`compatible ...`) .
- `paczka_medyczna` : z Rzeszowa przez Warszawę do Poznania – ciężarówka `tir_poznan` .
- `elektronika` i `czesci_samochodowe` : trasy przez `tir_poznan` / `tir_warszawa` do `rzeszow_magazyn` .

Długość i czas trwania

- **22 akcje** czasowe.
- **Makespan 95,002** – krytyczna ścieżka: turbina (żegluga 30 + TIR + pociąg 20) oraz równoległe dostawy pozostałych paczek.
- Planer **minimalizuje czas zakończenia**, nie liczbę kroków – stąd sens równoległego `load` i `drive` .
- **Długość planu** (22) vs **makespan** (95): przy durative to różne miary – ważniejszy jest czas końcowy.
- Samolot(`fly-plane` , duration 8) **nie** występuje w planie – brak korzystnej trasy lotniczej dla celów i ograniczeń `compatible` dla turbiny.

Wpływ topologii

Aspekt	Obserwacja
Węzeł Warszawa	<code>warszawa_lotnisko</code> jest centralnym punktem sieci drogowej – w planie większość przejazdów ciężarówek obejmuje ten węzeł
Morze	Skraca transport turbiny wzdłuż wybrzeża (Gdańsk-Szczecin) vs objazd lądem
Kolej	Jedyny sensowny sposób dostawy turbiny do <code>krakow_stacja</code> ze Szczecina
Lot	W tej instancji samolot nie pojawia się w planie – brak opłacalnej trasy dla celów
Ograniczenia <code>compatible</code>	Wymuszają wieloetapowy łańcuch dla dużego ładunku

4. Zadanie 2 – Robot odkurzający (15 pkt)

4.1. Opis problemu

Robot ma **odwiedzić** trzy pokoje i **sprzątnąć** każdy z nich. Model zgodny z treścią listy: predykaty `at` , `dirty` , `clean` ; akcje `move` i `clean` .

Stan początkowy: robot w `pokoj1` , wszystkie pokoje brudne.

Cel: `(and (clean pokoj1) (clean pokoj2) (clean pokoj3))` .

4.2. Plik `domain.pddl`

```
(define (domain vacuum)
  (:requirements :strips :typing)

  (:types robot room)

  (:predicates
    (at ?r - robot ?p - room)
    (dirty ?p - room)
    (clean ?p - room)
  )

  (:action move
    :parameters (?r - robot ?from - room ?to - room)
    :precondition (at ?r ?from)
    :effect (and
      (not (at ?r ?from))
      (at ?r ?to)
    )
  )

  (:action clean
    :parameters (?r - robot ?p - room)
    :precondition (and (at ?r ?p) (dirty ?p))
    :effect (and (not (dirty ?p)) (clean ?p))
  )
)
```

Uwaga modelowa: brak predykatu `(connected ?from ?to)` – robot może przejść **bezpośrednio** między dowolną parą pokoi (graf kompletny). Upraszcza to planowanie; w realnym świecie dodałoby się ograniczenie krawędzi.

4.3. Plik `problem.pddl`

```
(define (problem vacuum-three-rooms)
  (:domain vacuum)
```



```

(:objects
  robot - robot
  pokoj1 pokoj2 pokoj3 - room
)

(:init
  (at robot pokoj1)
  (dirty pokoj1)
  (dirty pokoj2)
  (dirty pokoj3)
)

(:goal (and
  (clean pokoj1)
  (clean pokoj2)
  (clean pokoj3)
))
)

```

4.4. Wygenerowany plan

Planer: **Pyperplan**, przeszukiwanie **BFS**.

```

(clean robot pokoj1)
(move robot pokoj1 pokoj2)
(clean robot pokoj2)
(move robot pokoj2 pokoj3)
(clean robot pokoj3)

```

Metryka	Wartość
Długość planu	5 akcji
Rozwinięte węzły	22
Czas planowania	~0,3 ms

Plik: `Zadanie-2/problem.pddl.soln`.

4.5. Analiza

Poprawność

- `clean` wymaga obecności robota i `(dirty ?p)` – nie można posprzątać zdalnie.

- Po każdym `clean` zachodzi `(clean ?p)` i nie ma `(dirty ?p)`.
- Po kroku 5 wszystkie trzy cele są spełnione.

Optymalność długości

- Każdy brudny pokój wymaga **dokładnie jednego** `clean` → minimum **3** akcji `clean`.
 - Robot startuje w `pokoj1`; musi być fizycznie w `pokoj2` i `pokoj3` → co najmniej **2** akcje `move`.
 - **Dolna granica: $3 + 2 = 5$** – plan BFS jest **optimalny**.
-

5. Zadanie 3 – Robot z dwoma ramionami (10 pkt)

5.1. Opis problemu

Robot z **dwoma ramionami** (`arm1`, `arm2`) przenosi **cztery piłki** z `room1` do `room2`. Dozwolone akcje: `move`, `pick-up`, `put-down`.

5.2. Domena i problem (zgodne z listą)

Fragment `domain.pddl`:

```
(:action pick-up
  :parameters (?r - robot ?a - arm ?b - ball ?rm - room)
  :precondition (and
    (at ?r ?rm)
    (inroom ?b ?rm)
    (arm-empty ?a)
  )
  :effect (and
    (holding ?a ?b)
    (not (arm-empty ?a))
    (not (inroom ?b ?rm))
  )
)
```

Cel w `problem.pddl`:

```
(:goal (and
  (inroom ball1 room2)
```

```
(inroom ball2 room2)
(inroom ball3 room2)
(inroom ball4 room2)
))
```

5.3. Wygenerowany plan

Zadanie-3/problem.pddl.soln :

```
(pick-up robot arm2 ball3 room1)
(pick-up robot arm1 ball1 room1)
(move robot room1 room2)
(put-down robot arm2 ball3 room2)
(put-down robot arm1 ball1 room2)
(move robot room2 room1)
(pick-up robot arm2 ball2 room1)
(pick-up robot arm1 ball4 room1)
(move robot room1 room2)
(put-down robot arm2 ball2 room2)
(put-down robot arm1 ball4 room2)
```

Metryka	Wartość
Długość planu	11
Rozwinięte węzły (BFS)	253
Czas planowania	~3 ms
Zinstancjonowane operatory	34

5.4. Analiza planu

Poprawność

- Kroki 1-2: dwa podniesienia – oba ramiona zajęte, piłki znikają z `room1` .
- Krok 3: `move` – robot zmienia pokój; piłki w ramionach.
- Kroki 4-5: `put-down` w `room2` .
- Krok 6: powrót po pozostałe piłki.
- Kroki 7-11: powtórzenie wcześniejszych ruchów – po zakończeniu wszystkie piłki w `room2` .

Wykorzystanie dwóch ramion

Robot może nieść **co najwyżej 2** piłki naraz → przy 4 piłkach potrzeba ≥ 2 kursów `room1` → `room2` .
Plan realizuje **dwa przemieszczenia po 2 piłki** – optymalna strategia równoległego użycia ramion.

Optymalność

Jedno pełne przemieszczenie: $2 \times \text{pick-up} + 1 \times \text{move} + 2 \times \text{put-down} = 5$ akcji.

Między wsadami: $1 \times \text{move}$ powrotny.

Razem: $5 + 1 + 5 = 11$ – dolna granica; BFS gwarantuje optimum.

6. Porównanie eksperymentów

Zadanie	Planer	Typ planu	Wynik	Stany	Czas planera
1 – Transport	OPTIC	durative	makespan 95,002 , 22 akcje	93	~0,15 s
2 – Odkurzacz	Pyperplan BFS	STRIPS	5 kroków	22	~0,3 ms
3 – Piłki	Pyperplan BFS	STRIPS	11 kroków	253	~3 ms

7. Wnioski końcowe

1. **PDDL** skutecznie oddziela wiedzę o działaniach (domena) od konkretnej instancji (problem), co ułatwia testowanie różnych scenariuszy transportu lub robota bez zmiany schematów akcji.
2. **STRIPS z typowaniem** (Zadania 2 i 3) jest wystarczający do klasycznych problemów sekwencyjnych; planer szybko znajduje optimum na małych przestrzeniach stanów.
3. **Akcje czasowe** modelują równoległość.
4. **Topologia i ograniczenia** (`compatible` , rodzaje połączeń) mają większy wpływ na plan niż liczba paczek – turbina wymusza łańcuch morze → droga → kolej.